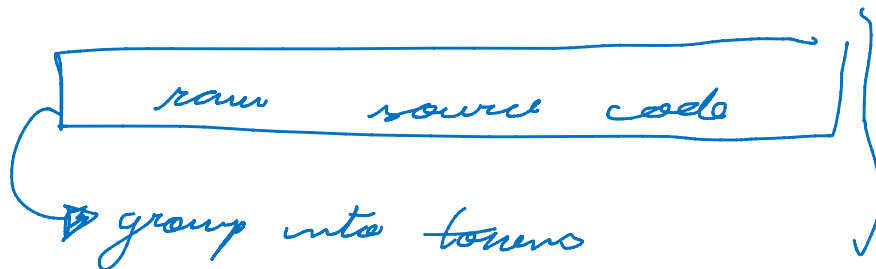


Scanner Implementation

sábado, 17 de maio de 2025 09:52

- Starting reading a source file
 - May need to reate one as examle so i can read it.
 - Already create one for the functional tests
 - Start with a strategy pattern to get the content out of the source files
 - This one will only ready and return the raw data as a string
 - Utility class that defines a class SourceReaderAsString
 - Define an interface ISourceReader
 - This configuration allows us to swap implementaton latter on
 - **Strategy Pattern** is when you:
 - Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.
 - You can swap file reading logic (e.g., for compressed files, cloud storage, etc.) without changing SourceScanner.
 - **Adheres to SOLID**
 - This follows:
 - **S**: Single Responsibility — Reader and Scanner are separate.
 - **O**: Open/Closed — You can extend with new readers.
 - **D**: Dependency Inversion — Scanner depends on an interface, not a concrete class.

Scanner



SWITCH

```
package com.craftinginterpreters.lox;
```

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.nio.charset.Charset;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.List;
```

```
public class Lox {
    public static void main(String[] args) throws IOException {
        if (args.length > 1) {
            System.out.println("Usage: jlox [script]");
            System.exit(64);
        } else if (args.length == 1) {
            runFile(args[0]);
        } else {
            runPrompt();
        }
    }
}
```

Receives an string as entry

I'll focus on this case

Given a source file,
how can I run this!!

```
private static void runFile(String path) throws IOException {
    byte[] bytes = Files.readAllBytes(Paths.get(path));
    run(new String(bytes, Charset.defaultCharset()));
}
```

Read everything as a string

Here is where everything starts

Both the prompt and the file runner are thin wrappers around this core function:

```
private static void run(String source) {
    Scanner scanner = new Scanner(source);
    List<Token> tokens = scanner.scanTokens();

    // For now, just print the tokens.
    for (Token token : tokens) {
        System.out.println(token);
    }
}
```

Create a new scanner class

given that class it creates a
list of tokens

Scanner for Conditional Match
Groups tokens per file

TOKEN

→ Represents a token

↳ maybe LF or something

SCANNER

→ scanTokens()